I want you to build a complete, modern, professional personal portfolio website for me from scratch.

Before writing any code, first understand who I am, what I have done, what my strengths are, and how I should be presented professionally.

My name is **Mazen Mahmoud Mousa**.

I am a **Computer & Artificial Intelligence student** with a strong interest in **Software Engineering, Full-Stack Web Development, .NET Development, and Artificial Intelligence**.

I do not want my portfolio to look like a generic student portfolio.

I want it to present me as a serious young software engineer who already builds real products, participates in competitions and hackathons, solves real-world problems, and combines software engineering with AI.

My main professional positioning should be:

Software Engineer | Full-Stack Developer | .NET Developer | AI Enthusiast

The main story behind the portfolio should be:

A software engineering student who builds real-world products, has hands-on experience with web development and .NET, uses AI to solve meaningful problems, and has participated successfully in hackathons and technical competitions.

Do not exaggerate my experience or make me look like a senior engineer.

Present me realistically as a high-potential student / junior software engineer with strong hands-on experience for my level.

IMPORTANT: WORKFLOW

Do NOT start coding immediately.

Before writing the actual website code, I want you to complete the following phases in order.

Phase 1 — Personal Brand Analysis

Analyze my profile and determine:

- My strongest professional positioning

- What makes me different from a typical student developer
- What recruiters should understand about me within the first 10–20 seconds
- Which projects should receive the most visual attention
- Which technical strengths should be emphasized
- What information should not dominate the portfolio

The portfolio should communicate that I:

- Build real products
- Solve real-world problems
- Have strong .NET and web development experience
- Work with both frontend and backend technologies
- Understand software architecture
- Use AI in meaningful projects
- Have practical automation experience
- Participate in hackathons and competitions
- Use Git and professional development workflows

Phase 2 — Information Architecture

Before coding, define the complete website structure.

Decide the best order and hierarchy for sections such as:

1. Hero
2. About Me
3. Technical Skills
4. Featured Projects
5. Other Projects
6. Experience / Training
7. Competitions & Achievements
8. Education

9. Certifications
10. GitHub / Coding Profile
11. Contact

Explain why each section exists and how the user should move through the portfolio.

The portfolio should tell a story rather than just display random cards.

Phase 3 — Design Direction

Before coding, define a complete visual direction.

Create a clear design concept including:

- Overall visual style
- Color palette
- Typography
- Spacing system
- Card style
- Button style
- Navigation style
- Project card style
- Section hierarchy
- Animation direction
- Mobile behavior
- Hover states
- Background treatment
- Border treatment
- Icon style

The visual direction should be:

- Modern
- Premium

- Clean
- Developer-focused
- Professional
- Minimal but not empty
- Visually impressive
- Smooth
- Suitable for recruiters
- Suitable for freelance clients
- Excellent on mobile

Avoid a generic Bootstrap student portfolio.

Avoid childish visuals.

Avoid excessive neon colors.

Avoid overusing gradients.

Avoid unnecessary visual effects.

A sophisticated dark theme or dark/light hybrid is preferred.

Use subtle accents and strong typography.

Phase 4 — Technical Architecture

Before building the UI, define the project architecture.

Explain:

- Folder structure
- Reusable page sections
- Data files
- Project data structure
- Skills data structure
- Constants
- Assets

- JavaScript modules
- Animation organization
- Responsive strategy

Do not hardcode project information repeatedly across the website.

Store project information, skill information, links, achievements, and other content inside clean reusable JavaScript data structures.

Only after finishing these four phases should you begin implementing the website.

PRIMARY TECHNOLOGY CHOICE

This portfolio should be built primarily using:

- **HTML5**
- **CSS3**
- **Vanilla JavaScript**

Do NOT use React, Vue, Angular, Next.js, Astro, or another frontend framework unless there is an extremely strong technical reason.

The portfolio does not need a heavy framework.

The goal is to create a highly polished static portfolio with:

- Excellent performance
- Very fast initial load
- Minimal JavaScript
- Great SEO
- Strong accessibility
- Easy maintenance
- Easy GitHub deployment
- Easy Cloudflare Pages deployment

The professional quality of the website should come from:

- Excellent UI/UX

- Strong typography
- High-quality layout
- Thoughtful spacing
- Strong visual hierarchy
- Beautiful project presentation
- Subtle animation
- Good responsive behavior
- Attention to detail

Do NOT assume that a framework is required to make the website look professional.

The final website should prove that a well-engineered vanilla frontend can still feel premium and modern.

Use third-party libraries only when they provide a clear benefit.

Possible lightweight additions:

- Lucide Icons
- GSAP for advanced but subtle animation if needed
- AOS only if it is actually useful

Prefer native CSS transitions and modern browser APIs whenever possible.

Avoid unnecessary dependencies.

TARGET VISITOR EXPERIENCE

This portfolio will primarily be opened by:

- Recruiters
- Hiring managers
- Software engineers
- Potential freelance clients

Many visitors will reach the website from my **LinkedIn profile**.

Optimize the entire experience for first-time visitors.

Within approximately **10–20 seconds**, the visitor should understand:

- Who I am
- What type of developer I am
- What technologies I work with
- What my strongest projects are
- That I build real-world products
- That I have competition and hackathon experience
- How to view my work
- How to contact me

Do not make users search for important information.

Navigation should be extremely simple and intuitive.

The portfolio must feel smooth and effortless on both mobile and desktop.

TECHNICAL SKILLS

My strongest technical skills include:

- HTML
- CSS
- JavaScript
- React
- Tailwind CSS
- C#
- ASP.NET Core
- Entity Framework Core
- SQL Server
- REST APIs

- Clean Architecture
- Git
- GitHub
- Python
- Excel Automation
- UI Automation
- PyInstaller
- Vite
- GitHub Pages
- Netlify
- Vercel
- Cloudflare Pages

I also have experience with modern frontend tools and concepts such as:

- React Router
- Axios
- TanStack Query
- Framer Motion
- shadcn/ui
- Responsive Design
- Mobile-First Design
- UI/UX thinking
- Arabic/English interfaces

Important:

Even though the portfolio itself should be built with HTML, CSS, and Vanilla JavaScript, still show technologies such as React and Tailwind in my Skills section because they are technologies I use in other projects.

FEATURED PROJECTS

1. IRIS

IRIS is one of my most important projects and should receive the strongest visual emphasis in the portfolio.

IRIS is an AI-powered educational platform focused on children with special educational needs, especially children with:

- Down Syndrome
- Williams Syndrome

The main target age range is approximately **6 to 12 years old**.

The project addresses two major problems.

Problem 1 — Education

Children with very different conditions and educational needs may sometimes be grouped under broad inclusion systems even though their needs can be very different.

This can lead to educational experiences that are not personalized enough.

IRIS aims to provide a more personalized learning experience.

Problem 2 — Parents

Many parents may struggle to understand:

- How to teach their children
- How to communicate with them
- How to support their development
- What learning methods are most suitable
- How to follow progress
- How to deal with challenges

This can create a gap between parents and their children.

IRIS aims to help parents understand the right approach through structured guidance, expert knowledge, and community support.

Main Features

IRIS can include:

- AI-generated personalized learning roadmaps
- Personalized learning recommendations
- Educational stories
- Educational games
- Letters and numbers learning
- Progress tracking
- Parent dashboard
- Parent guidance
- Expert recommendations
- Parent community
- Progress analytics
- Adaptive educational content

The personalized learning roadmap can use information such as:

- Age
- Abilities
- Interests
- Hobbies
- Learning level
- Assessment results
- Personal preferences

Achievements

IRIS achieved:

- **2nd Place in a GDG Hackathon at the Egyptian Chinese University**
- The hackathon included **more than 100 teams**
- IRIS reached the **Top 6 in an IEEE-related conference or competition**

- IRIS was also presented or participated in a competition related to the **Engineers Syndicate**

Present these achievements professionally without exaggeration.

IRIS should probably be the first featured project.

---

2. TeacherHub

TeacherHub is a SaaS platform designed to help teachers manage their educational work.

It is a larger system that demonstrates my backend development and software architecture skills.

The system uses a multi-tenant approach with:

- A single database
- TeacherId-based data isolation

Main modules include:

- Students
- Groups
- Exams
- Grades
- Attendance
- Settings
- WhatsApp messaging

Technical architecture includes:

- ASP.NET Core 8
- Clean Architecture
- Domain Layer
- Application Layer
- Infrastructure Layer
- API Layer

- Entity Framework Core
- SQL Server
- JWT Authentication
- Serilog
- React frontend
- Arabic / English support
- Theme provider
- Soft delete
- Mobile responsive interface

TeacherHub should demonstrate that I understand how larger real-world systems are structured.

The portfolio should highlight concepts such as:

- Architecture
- Maintainability
- Scalability
- Data isolation
- Backend APIs
- Authentication
- Full-stack development

---

3. A-SCALE PLU Updater

A-SCALE PLU Updater is a Windows automation tool created to solve a real business problem.

It automates updating PLU information inside A-SCALE software.

The tool can update:

- Product names
- Prices

- Product codes

The information comes from Excel and is automatically transferred into the A-SCALE software.

Technologies and concepts used include:

- Python
- Excel processing
- openpyxl
- UI Automation
- pywinauto
- ctypes
- Win32 messaging
- PyInstaller

The project was packaged as a Windows executable.

The final project went through approximately **312 tests**.

This project should be used to demonstrate that I can solve practical operational problems through software automation, not only build websites.

---

4. Shadow Rooftop

Shadow Rooftop is a professional digital menu website created for a rooftop venue with a pyramids view.

The website is designed mainly for customers opening it through a QR code on their phones.

Important characteristics:

- Mobile-first
- Fast loading
- Premium visual identity
- Golden luxury theme

- Responsive design
- Menu categories
- Breakfast pages
- Strong photography presentation
- Easy navigation
- Tourist-friendly interface

The project was deployed using Cloudflare Pages.

This project should demonstrate my frontend development and UI/UX skills.

---

5. KYND Café Menu

A digital café menu built using:

- HTML
- CSS
- JavaScript

The focus was:

- Simplicity
- Fast loading
- Responsive design
- Mobile-first experience
- QR-based access
- Clear category navigation

It was deployed online.

Use this project as another example of practical frontend development.

---

6. Home-and-Car

Home-and-Car is a service platform concept inspired by the convenience of apps such as Uber, but focused on providing multiple service categories.

The platform can include:

- Cars
- Technical services
- Service selection
- WhatsApp communication
- Arabic / English language support
- Responsive design
- Mobile-first UX

This project should demonstrate product thinking and service-oriented UX.

---

7. CodeForge-Studio / AI Video Editing Engine

CodeForge-Studio is an AI-based video editing system developed through multiple milestones.

The goal is to explore how AI can make editing decisions and assist with video production.

The system includes concepts such as:

- Automated storytelling
- AI caption systems
- Editing automation
- AI-based editing decisions
- Camera direction logic
- Video structure understanding

One of the later development milestones was:

AI Video Editor Milestone 15: Camera Direction Engine

This project should show my interest in experimental AI systems and advanced software development.

Do not present unfinished concepts as production-ready features.

Clearly differentiate between:

- Implemented functionality
- Experimental functionality
- Planned functionality

GIT & GITHUB

I actively use Git and GitHub in my projects.

My workflow includes:

- git status
- git add
- git commit
- git push
- Branches
- Pull Requests
- Repository management
- Conventional Commit messages

Some of my repositories include:

- KYND
- Home-and-Car
- Shadow Rooftop
- TeacherHub
- CodeForge-Studio

Create a section or subtle visual area that demonstrates that I use professional version-control workflows.

Do not create a giant Git command tutorial.

EDUCATION & TRAINING

University

I am a student in the:

Faculty of Computers & Artificial Intelligence

Present this cleanly inside the Education section.

Digital Egypt Pioneers Initiative — DEPI

I am enrolled in the **.NET Development / C# track**.

Relevant topics include:

- C#
- .NET
- SQL
- Database design
- ERD
- Backend development
- Software development practices

DEPI should appear in either Training, Experience, or Education depending on what creates the best information architecture.

Freelancing Training

I completed or participated in a freelancing training/program related to **ITIDA Gigs**.

This can appear under Training / Certifications depending on the available details.

COMPETITIONS & ACHIEVEMENTS

Highlight these achievements:

- **2nd Place — GDG Hackathon at the Egyptian Chinese University**
- More than 100 teams participated
- Project: IRIS
- **Top 6 — IEEE-related conference / competition with IRIS**
- Participated in a competition related to the **Engineers Syndicate**
- Participated in the **ITIDA-SECC Software Testing Competition**

Create a professional achievements section.

Do not use exaggerated phrases.

Keep everything factual.

HERO SECTION

The Hero section is extremely important.

Do not use generic sentences such as:

“Hello, I am Mazen and I am passionate about coding.”

The hero should immediately communicate:

- Software engineering
- Real-world projects
- .NET
- Modern web technologies
- AI
- Product building

Possible direction:

Software Engineer building real-world products with .NET, modern web technologies, and AI.

You may improve this sentence and create something stronger.

Add a short secondary description that explains that I am a Computer & AI student building practical software products and participating in technical competitions.

Possible CTAs:

- View My Work
- Download CV
- GitHub
- Contact Me

Use placeholders until I provide the actual links.

ABOUT SECTION

The About section should be concise.

Do not write a huge biography.

It should communicate that:

- I am still a student
- I already have practical development experience
- I work on real products
- I build frontend and backend systems
- I have automation experience
- I explore AI-based systems
- I participate in hackathons and competitions

The tone should be confident but realistic.

SKILLS SECTION

Do not create a giant wall of technology logos.

Organize skills intelligently.

Possible groups:

Frontend

- React

- JavaScript
- HTML
- CSS
- Tailwind CSS

Backend

- C#
- ASP.NET Core
- REST APIs
- Entity Framework Core

Database

- SQL Server
- Database Design

Architecture

- Clean Architecture
- Authentication
- Multi-Tenant Systems

Tools

- Git
- GitHub
- Vite
- Visual Studio
- VS Code

Automation

- Python
- Excel Automation
- UI Automation

- PyInstaller

Concepts

- Responsive Design
- Mobile-First
- UI/UX
- Arabic / English Interfaces

Use visual hierarchy to show my strongest technologies first.

PROJECT PRESENTATION

Projects should be the main focus of the website.

For major projects such as IRIS and TeacherHub, use larger project sections/cards.

Each project can include:

- Project image
- Name
- Short description
- Problem solved
- My role
- Key features
- Technologies
- Achievements if relevant
- GitHub button
- Live Demo button
- Case Study button if appropriate

For smaller projects, use more compact project cards.

Make it easy to add more projects later.

USER EXPERIENCE REQUIREMENTS

The website must be extremely easy to use.

A visitor should never feel lost.

Use:

- Simple navigation
- Clear section labels
- Obvious calls to action
- Smooth scrolling
- Clear project hierarchy
- Easy-to-find GitHub and LinkedIn links
- Easy access to CV
- Clear contact options

Recommended navigation direction:

- Home
- Work
- About
- Experience
- Contact

Keep it simple.

Do not overcrowd the navigation.

DESIGN REQUIREMENTS

The website should feel:

- Modern
- Premium
- Professional

- Technical
- Clean
- Smooth
- High-end
- Developer-focused
- Recruiter-friendly

Avoid:

- Generic templates
- Bootstrap-like student portfolios
- Huge gradient backgrounds everywhere
- Excessive glassmorphism
- Excessive neon
- Too many glowing effects
- Overloaded animations
- Too many floating shapes
- Cheap-looking cards

Use:

- Excellent typography
- Strong spacing
- Clean hierarchy
- Elegant cards
- Subtle animations
- Premium hover effects
- Good whitespace
- Clear navigation
- High-quality project presentation

The fact that the website uses HTML/CSS/JavaScript should NOT make it visually simple.

The visual execution should feel like a high-end custom portfolio.

CSS ARCHITECTURE

Keep the CSS structured and maintainable.

Suggested structure:

- variables.css
- reset.css
- base.css
- layout.css
- components.css
- animations.css
- responsive.css

Or another similarly clean architecture.

Use CSS custom properties for:

- Colors
- Font sizes
- Spacing
- Border radius
- Shadows
- Container widths
- Animation timings

Example concept:

```
:root {
  --bg-primary: ...;
  --bg-secondary: ...;
```

--text-primary: ...;

--text-muted: ...;

--accent: ...;

--border: ...;

--radius-md: ...;

--container-width: ...;

}

Do not scatter random values everywhere.

JAVASCRIPT ARCHITECTURE

Keep JavaScript modular.

Possible structure:

- main.js
- data/projects.js
- data/skills.js
- data/achievements.js
- modules/navigation.js
- modules/animations.js
- modules/projects.js

Use ES modules where appropriate.

Avoid large monolithic JavaScript files.

Do not use JavaScript for things that CSS can handle cleanly.

ANIMATIONS

Animations should support the experience, not distract from it.

Good examples:

- Section reveal
- Small card hover animations
- Button interactions
- Hero entrance animation
- Smooth project transitions
- Navigation interactions
- Subtle image movement

Use native CSS animations and transitions where possible.

If a more advanced animation requires a library, GSAP can be considered.

Avoid:

- Constant movement
- Excessive parallax
- Huge page transition effects
- Animations that reduce performance
- Heavy animation libraries without a clear reason

RESPONSIVE DESIGN

The entire website must be designed mobile-first.

Test the layout for:

- Small phones
- Large phones
- Tablets
- Laptops
- Large desktop screens

Important:

Do not simply shrink the desktop version.

Mobile should have its own thoughtful layout decisions.

Project cards, typography, navigation, images, spacing, and buttons should all adapt correctly.

PERFORMANCE

Performance is one of the main reasons for using a Vanilla frontend.

Prioritize:

- Minimal JavaScript
- Optimized assets
- Lazy-loaded images
- Responsive images
- Proper width and height attributes
- Modern image formats where appropriate
- Efficient animations
- No unnecessary dependencies
- Preloading only critical resources
- Deferred JavaScript
- Optimized font loading

The website should feel almost instant on a good connection.

Aim for strong Lighthouse performance.

SEO

Add proper SEO foundations including:

- Page title
- Meta description
- Canonical placeholder

- Open Graph metadata
- Twitter/X social metadata
- Proper heading hierarchy
- Semantic HTML
- Favicon placeholder
- robots.txt if appropriate
- sitemap.xml if useful
- Structured data if appropriate

The portfolio link may be shared on LinkedIn, so social preview metadata is important.

ACCESSIBILITY

Use:

- Semantic HTML
- Accessible buttons and links
- Keyboard navigation
- Visible focus states
- Sufficient color contrast
- Alt text
- Accessible mobile navigation
- aria attributes only where useful

Do not sacrifice accessibility for visual design.

GITHUB AND DEPLOYMENT

The source code will be uploaded to GitHub.

Keep the repository clean and professional.

Include:

- README.md
- .gitignore
- Clear folder structure
- Meaningful file names

The project should be deployable easily to:

- GitHub Pages
- Cloudflare Pages

Because this is a static HTML/CSS/JavaScript website, deployment should remain simple.

Do not introduce a build process unless there is a clear benefit.

Ideally:

git push → deployment updates automatically

RECOMMENDED PROJECT STRUCTURE

Do not blindly use this structure if you have a better architecture, but use it as a reference:

```
portfolio/
│
├── index.html
│
├── css/
│   ├── variables.css
│   ├── reset.css
│   ├── base.css
│   ├── layout.css
│   ├── components.css
│   ├── animations.css
│   └── responsive.css
```

```
│
├── js/
│   ├── main.js
│   │
│   ├── data/
│   │   ├── projects.js
│   │   ├── skills.js
│   │   ├── achievements.js
│   │   └── socials.js
│   │
│   └── modules/
│       ├── navigation.js
│       ├── animations.js
│       └── projects.js
│
├── assets/
│   ├── images/
│   │   ├── profile/
│   │   ├── projects/
│   │   └── achievements/
│   │
│   ├── icons/
│   └── documents/
│       └── Mazen-Mahmoud-CV.pdf
│
```

├── favicon/

├── README.md

└── .gitignore

Keep the actual architecture practical.

Do not split tiny amounts of code across dozens of unnecessary files.

CONTENT DATA ARCHITECTURE

Project information should be data-driven where practical.

Example:

```
export const projects = [
 {
  id: "iris",
  title: "IRIS",
  category: "AI / Education",
  shortDescription: "",
  description: "",
  problem: "",
  solution: "",
  role: "",
  technologies: [],
  features: [],
  image: "",
  githubUrl: "",
  liveUrl: "",
  caseStudyUrl: "",
  featured: true,
```

```
  achievement: "",
  status: "active"
 }
];
```

The purpose is to make adding or editing projects easy later.

Do not overengineer it.

MISSING CONTENT

I may not provide everything immediately.

Examples:

- Screenshots
- GitHub URLs
- LinkedIn URL
- Email
- CV
- Certificates
- Live project links

Do not invent them.

Use obvious placeholders.

Keep them centralized so they are easy to replace.

README

Create a professional README explaining:

- Portfolio overview
- Technologies used
- Folder structure

- How to run the site locally
- How to update my details
- How to add projects
- How to replace project images
- How to update social links
- How to add my CV
- How to update SEO metadata
- How to deploy using GitHub Pages
- How to deploy using Cloudflare Pages

FINAL REVIEW

Before considering the project complete, review the website from the perspective of:

Recruiter

Can they quickly understand what I do?

Hiring Manager

Are my strongest projects obvious?

Software Engineer

Does the repository look clean and intentional?

Potential Client

Does the portfolio create confidence that I can deliver polished software?

Mobile User

Is the site excellent on a phone?

Performance

Does the site load and respond quickly?

Visual Quality

Does it look custom and premium instead of like a student template?

Accessibility

Can keyboard and screen-reader users navigate the important content?

Fix weak areas before finishing.

FIRST RESPONSE — DO NOT CODE YET

For your first response, do not generate website files.

First give me:

1. **Personal Brand Analysis**
2. **Recommended Portfolio Story**
3. **Final Site Information Architecture**
4. **Design Concept**
5. **Color Palette**
6. **Typography System**
7. **UI / Component Style**
8. **Animation Strategy**
9. **Responsive Strategy**
10. **Recommended Vanilla Frontend Architecture**
11. **Folder Structure**
12. **Content Data Architecture**
13. **Which projects should be featured vs secondary**
14. **Recruiter-first UX flow**
15. **Performance strategy**
16. **A clear implementation roadmap**

Only after this architecture and design plan is established should you begin writing the actual code.

Do not rush directly into implementation.

Think like both a **senior product designer and a senior frontend engineer** while designing the system, but present me accurately as a **student / junior software engineer**, not a senior developer.

The final goal is:

A fast, premium, highly professional personal portfolio built with clean HTML, CSS, and Vanilla JavaScript that looks impressive to recruiters, hiring managers, software engineers, and potential clients while remaining simple to use, fast to load, easy to maintain, and easy to deploy.